

ECOM EXPERTS

Ecom uploadgithub

Manual de usuario, tecnico y de desarrollo

ecom_uploadgithub 1.0.0 · 07/09/2026

Contenido

¿Qué hace este módulo?

Lo que NUNCA se sube (por seguridad)

Instalación

Configuración (paso a paso, también dentro del módulo)

Programar el backup automático

Primera copia

Comprobaciones

Preguntas frecuentes

Objetivo

Arquitectura

Flujo de backup

Seguridad

Claves de Configuración (tabla configuration)

Compatibilidad

Convenciones aplicadas

Puntos de extensión

Manifiesto incremental

Reintentos / lotes

Errores y logs

Pruebas recomendadas

¿Qué hace este módulo?

Guarda una copia de seguridad del **código** de tu tienda en un repositorio **privado** de GitHub y la mantiene actualizada automáticamente. **Nunca** sube contraseñas, base de datos ni ficheros de configuración sensibles.

Lo que NUNCA se sube (por seguridad)

`app/config`, `parameters.php`, `.env`, `settings.inc.php`, certificados (`.pem`, `.key`), la carpeta `/var`, caché, logs e imágenes/media. Aunque alguien accediera a tu repositorio, **no encontraría credenciales**.

Instalación

1. Sube la carpeta `ecom_uploadgithub` a `/modules` (o instala el ZIP).
2. En el back office: **Módulos** > **ecom_uploadgithub** > **Instalar**.
3. Pulsa **Configurar**.

Configuración (paso a paso, también dentro del módulo)

1. **Crea una cuenta** en <https://github.com/signup> y verifica tu email.
2. **Activa 2FA** en *Settings* > *Password and authentication* (obligatorio).
3. **Crea un repositorio** nuevo y marca **Private** (nunca Public). Ej.: `backup-tienda`.
4. **Crea un token** en *Settings* > *Developer settings* > *Fine-grained tokens*: - Acceso: **solo** ese repositorio. - Permiso **Contents: Read and write**. - Copia el token (`github_pat_...`).
5. En la configuración del módulo rellena: **Owner**, **Repositorio**, **Rama** (main) y pega el **Token** (se guarda cifrado).
6. **Activa el backup** y pulsa **Guardar**.

Programar el backup automático

Copia la **URL del cron** que aparece arriba en la pantalla de configuración y crea una tarea programada en tu hosting (cPanel/Plesk) o crontab para que la llame cada hora:

```
0 * * * * curl -s "https://tutienda.com/module/ecom_uploadgithub/cron?token=TU_TOKEN" >/dev/null 2>&1
```

Primera copia

Pulsa **Ejecutar backup ahora** para hacer la primera subida y comprobar que funciona.

Comprobaciones

- En GitHub deberías ver tu código, **pero nunca** `app/config` ni `parameters.php`.
- El panel muestra **Última ejecución** y **Último resultado**.

Preguntas frecuentes

- **¿Es 100% seguro?** Ningún sistema lo es al 100%, pero lo más importante es que los secretos **no se suben**, así que no hay nada que robar. El repositorio es privado y solo tú (y quien invites) podéis verlo.
- **¿Y si pongo el repo en público por error?** El módulo lo detecta y **aborta** el backup automáticamente.
- **¿Consume mucho?** No: solo sube los ficheros que han cambiado desde la última copia.

Ecom Upload GitHub — Documentación Técnica

Autor: Ecom Experts (ecomyseo@gmail.com) · **Licencia:** AFL-3.0 · **Versión:** 1.0.0

Objetivo

Backup automático e incremental del **código** de una tienda PrestaShop a un repositorio **privado** de GitHub, excluyendo de forma forzada todos los ficheros sensibles.

Arquitectura

ecom_uploadgithub.php	Clase principal (config, install, getContent)
classes/	
Logger.php	(Ecom_Ugh_Logger) Log opcional en /logs si ECOM_UGH_DEBUG=1
Crypto.php	(Crypto) Cifrado AES-256-CBC del token (OpenSSL nativo)
SecurityFilter.php	(SecurityFilter) Lista negra blindada de exclusiones
GithubClient.php	(GithubClient) Cliente API GitHub vía cURL (Contents API)
BackupEngine.php	(BackupEngine) Recorrido + hash git-blob + subida incremental
controllers/front/cron.php	Endpoint cron protegido por token
views/templates/admin/status.tpl	Panel de estado + guía de configuración
logs/	Logs + manifest.json (no se suben)

Flujo de backup

1. El cron del servidor llama a `.../module/ecom_uploadgithub/cron?token=XXXX`.
2. `cron.php` valida el token con `hash_equals` (timing-safe). Sin token → 403.
3. `BackupEngine::run()` : - Verifica `ECOM_UGH_ENABLED`, owner, repo y token. - **Blindaje:** `GithubClient::isRepoPrivate()` — si el repo no es privado, **aborta**. - Recorre `_PS_ROOT_DIR_` con `RecursiveDirectoryIterator`. - Por cada fichero aplica `SecurityFilter::isExcluded()`. - Calcula el hash git-blob: `sha1("blob <len>\0<content>")`. - Compara con `logs/manifest.json`. Si cambió, sube vía Contents API (PUT) usando el `sha` previo para actualizar. - Respeta `ECOM_UGH_MAX_FILES` (lotes) y `ECOM_UGH_MAX_SIZE_MB`.
4. Actualiza manifiesto, `ECOM_UGH_LAST_RUN` y `ECOM_UGH_LAST_STATUS`.

Seguridad

- **Token cifrado** AES-256-CBC con clave aleatoria (`ECOM_UGH_ENC_KEY`).
- **Exclusión forzada** (no configurable por el usuario): `app/config/`, `config/`, `var/`, `cache/`, `parameters.php`, `.env`, `settings.inc.php`, `*.pem/.key/.crt/.p12`, `.htpasswd`, `img/`, `logs`, `backups`, `.git/`.
- **Verificación de repo privado** antes de cada subida.

- **Token de cron** de 48 caracteres, comparación timing-safe.
- Carpeta `logs/` protegida con `.htaccess` (Deny from all).

Claves de Configuración (tabla configuration)

`ECOM_UGH_ENABLED`, `ECOM_UGH_DEBUG`, `ECOM_UGH_OWNER`, `ECOM_UGH_REPO`, `ECOM_UGH_BRANCH`, `ECOM_UGH_TOKEN` (cifrado), `ECOM_UGH_EXTRA_EXCLUDE`, `ECOM_UGH_MAX_FILES`, `ECOM_UGH_MAX_SIZE_MB`, `ECOM_UGH_ENC_KEY`, `ECOM_UGH_CRON_TOKEN`, `ECOM_UGH_LAST_RUN`, `ECOM_UGH_LAST_STATUS` .

Compatibilidad

PHP 7.4–8.x · PrestaShop 1.6 / 8 / 9 · Sin Composer · Funciones nativas (cURL, OpenSSL).

Ecom Upload GitHub — Guía de Desarrollo

Autor: Ecom Experts · **Licencia:** AFL-3.0

Convenciones aplicadas

- Clase principal `Ecom_Uploadgithub` (sin namespace, global namespace) que coincide con la carpeta `ecom_uploadgithub`.
- `if (!defined('_PS_VERSION_')) exit;` tras la cabecera en todos los `.php`.
- `index.php` recursivo en cada carpeta con redirección.
- Carga manual con `require_once`, cero dependencias de Composer.
- `isUsingNewTranslationSystem() => true`.

Puntos de extensión

- **Añadir exclusiones permanentes:** edita los arrays `$dirPrefixes`, `$exactNames`, `$extensions`, `$contains` en `classes/SecurityFilter.php`.
- **Cambiar estrategia de subida:** `GithubClient` usa la Contents API (1 PUT por fichero). Para grandes volúmenes se puede migrar a la Git Data API (blobs + tree + commit + ref) en un solo commit.
- **Programación interna:** actualmente el disparo es vía URL+token (cron real del servidor). Para usar el cron de PrestaShop, registrar el hook `actionCronJob` o el módulo `cronjobs`.

Manifiesto incremental

`logs/manifest.json` mapea `ruta_relativa => sha_git_blob`. La detección de cambios es local (sin llamar a GitHub), por lo que una pasada sin cambios no consume cuota de API salvo la verificación de repo privado.

Reintentos / lotes

`ECOM_UGH_MAX_FILES` limita ficheros por pasada para no agotar el `max_execution_time`. Lo no procesado se sube en la siguiente ejecución del cron.

Errores y logs

Activa **Modo depuración** en configuración. Se generan `logs/backup-YYYY-MM-DD.log`. Los eventos críticos (repo no privado, fallo de subida) se registran siempre (`force=true`).

Pruebas recomendadas

1. `php -l` sobre todos los `.php` (sin errores de sintaxis).
2. Backup manual desde el botón "Ejecutar backup ahora".
3. Verificar en GitHub que **no** aparecen `app/config`, `parameters.php`, `.env`, `var/`.
4. Forzar repo público temporalmente y comprobar que el backup **se aborta**.